Smista Rifiuti

Il gioco in GDScript (Godot 4), spiegato riga per riga

Una guida per capire come è costruito il gioco: ogni riga di codice è affiancata da una spiegazione di cosa significa e perché è scritta così. È il porting in GDScript della versione web (HTML + JavaScript) che abbiamo fatto insieme.

Indice

- **1.** Come usare questa guida e preparare il progetto in Godot
- **2.** GDScript in pillole (e confronto con il JavaScript)
- **3.** La mappa dei file
- **4.** Parte 1 · word_gen.gd — il generatore dei rifiuti
- **5.** Parte 2 · save_manager.gd — salvataggi e Pokédex
- **6.** Parte 3 · person_drawer.gd — disegnare l'operaio
- **7.** Parte 4 · game.gd — il gioco
- **8.** Cosa manca e come proseguire (esercizi)

Nota onesta: la sintassi di tutti i file è stata controllata con un parser GDScript, ma qui non ho potuto eseguire il gioco dentro Godot. Se Godot ti segnala un errore, copia il messaggio e lo sistemiamo insieme: è un ottimo modo per imparare.

1 · Come usare questa guida e preparare il progetto

I file `.gd` completi si trovano nella cartella `godot_port/scripts` del repository: copiali da lì invece di ricopiarli dal PDF. Nelle parti 1-4 la tabella ha tre colonne: il numero di riga nel file, il codice e la spiegazione. I colori del codice: parole chiave, testo tra virgolette, numeri, commenti.

Preparare il progetto (Godot 4.2 o successivo)

- Scarica **Godot 4.x** (non la 3.x: la sintassi è diversa) e crea un **nuovo progetto**. Il renderer "Compatibility" va benissimo per un gioco 2D.
- Crea la cartella `scripts` nel progetto e metti dentro i 4 file: `word_gen.gd`, `save_manager.gd`, `person_drawer.gd`, `game.gd`.
- **Autoload:** Progetto → Impostazioni progetto → Globali → Autoload. Aggiungi `res://scripts/save_manager.gd` con il nome `SaveManager`. (Gli altri due script, WordGen e PersonDrawer, si trovano da soli grazie a `class_name`.)
- **Scena principale:** crea una scena con un solo nodo **Node2D**, attacca `game.gd`, salvala come `main.tscn` e impostala come scena principale (Impostazioni progetto → Applicazione → Esecuzione).
- **Finestra:** Impostazioni progetto → Display → Window: larghezza 960, altezza 540 (le stesse misure usate nel codice).
- Premi **F5**. Comandi: A/D o frecce per muoverti, Spazio per prendere e lasciare, H per assumere un collega, P per la pausa.

2 · GDScript in pillole

GDScript è il linguaggio di Godot. Assomiglia molto a Python: si indenta con TAB (o spazi, ma sempre uguali), non servono punti e virgola e i blocchi si aprono con i due punti. Ecco i concetti che incontrerai nel codice.

Concetto	Esempio	Che cosa significa
Variabile	`var score := 0`	var crea una variabile. ":=" fa dedurre il tipo da Godot (qui un intero). Con ": int = 0" il tipo lo scrivi tu.
Costante	`const W := 960.0`	Valore che non cambia mai. Per convenzione si scrive in MAIUSCOLO.
Tipi principali	`int, float, bool, String, Color, Vector2, Rect2`	Numeri interi e decimali, vero/falso, testo, colore, coppia (x, y), rettangolo.
Funzione	`func somma(a: int, b: int) -> int:`	Definisce una funzione con parametri tipizzati e tipo di ritorno dopo "->". "-> void" = non ritorna nulla.
Funzione statica	`static func make_word(...)`	Si chiama sulla classe (WordGen.make_word) senza creare un oggetto.
Nome di classe	`class_name WordGen`	Rende lo script raggiungibile da ovunque con quel nome.
Ereditarietà	`extends Node2D`	Lo script "è" un Node2D e ne eredita funzioni e proprietà.
Lista (Array)	`var l := [1, 2, 3] · l.append(4)`	Elenco ordinato. Si legge con l[0]; l.size() dà la lunghezza; l[-1] è l'ultimo.

Concetto	Esempio	Che cosa significa
Dizionario	`var d := {"x": 5} · d["x"]`	Coppie chiave: valore. d.get("y", 0) dà 0 se la chiave non esiste; d.has("x") controlla.
Condizioni	`if / elif / else`	Come in Python. Gli operatori logici si scrivono and, or, not.
Cicli	`for i in range(5): · while ...:`	for scorre liste, dizionari (le chiavi), stringhe e range(). break esce, continue salta al giro dopo.
Ternario	`a if cond else b`	Sceglie tra due valori: "a" se cond è vera, altrimenti "b".
Formattazione	`"%d punti" % 5 · "%s %d" % [a, b]`	% inserisce valori nel testo: %d intero, %s testo, %.1f decimale con un decimale.
Funzione anonima	`func(a, b): return a < b`	Piccola funzione senza nome (lambda), usata per ordinare o filtrare.
Commenti	`# nota · ## nota di documentazione`	Tutto dopo il # viene ignorato da Godot.

Le funzioni speciali che Godot chiama per te

Funzione	Quando viene chiamata	A cosa serve nel gioco
`_ready()`	Una volta, quando il nodo entra in scena.	Preparazione iniziale: carica il salvataggio, registra i tasti.
`_process(delta)`	A ogni fotogramma. delta = secondi dall'ultimo fotogramma.	Tutta la logica: movimento, arrivo delle parole, intelligenza dei colleghi.
`_draw()`	Quando serve ridisegnare (dopo queue_redraw()).	Disegna sfondo, casse, personaggi, cartelli, punteggi.
`_unhandled_input(event)`	Per ogni evento di input non gestito altrove.	Prendi/lascia, assumi, pausa, click sul riquadro.

Dal JavaScript al GDScript

Se hai in mente la versione web, ecco come si traducono le cose che ricorrono più spesso.

JavaScript (versione web)	GDScript (Godot)	Nota
`let x = 5 / const K = 5`	`var x := 5 / const K := 5`	Stesso significato.
`function f(a) { ... }`	`func f(a) -> void:`	Niente graffe: conta l'indentazione.
`=== e !==`	`== e !=`	In GDScript basta un "==".
`arr.push(x)`	`arr.append(x)`	
`arr.filter(w => w.x > 0)`	`arr.filter(func(w): return w["x"] > 0)`	La lambda si scrive con "func".
`obj.text / obj["text"]`	`obj["text"]`	I Dictionary si leggono con le parentesi quadre.
`Math.floor / Math.round`	`floori() / roundi() (floorf per i decimali)`	Le funzioni hanno suffisso: i = intero, f = decimale.

JavaScript (versione web)	GDScript (Godot)	Nota
`Math.min / Math.max`	`minf, maxf, mini, maxi, clampf, clampi`	clamp limita un valore tra un minimo e un massimo.
`` `Punti ${score}` ``	`"Punti %d" % score`	Si usa % invece dei backtick.
`for (const w of words)`	`for w in words:`	
`requestAnimationFrame(loop)`	`func _process(delta):`	Godot chiama la funzione da solo, ad ogni fotogramma.
`ctx.fillRect(x, y, w, h)`	`draw_rect(Rect2(x, y, w, h), colore)`	Solo dentro _draw().
`ctx.arc(...) + fill()`	`draw_circle(centro, raggio, colore)`	Per archi: draw_arc().
`ctx.fillText(t, x, y)`	`draw_string(font, Vector2(x, y), t, ...)`	Serve un font; si usa ThemeDB.fallback_font.
`addEventListener("keydown")`	`InputMap + _unhandled_input(event)`	Si definiscono "azioni" (es. "act") legate a più tasti.
`localStorage.setItem(...)`	`FileAccess + JSON.stringify(...)`	I dati vanno in un file dentro "user://".
`Seed + mulberry32 (js/core.js)`	`RandomNumberGenerator con .seed`	Stesso seme → stessa sequenza di numeri.

3 · La mappa dei file

Il gioco web era diviso in file con compiti precisi. Nel porting abbiamo mantenuto la stessa idea.

File GDScript	Che cosa fa	Equivale a (web)
`word_gen.gd`	Genera le parole palindrome dal seme, decide tipo/colore, punti e peso. Nessun disegno.	`js/core.js`
`save_manager.gd`	Salvataggi multipli, Pokédex generale, esporta/importa. È un Autoload.	`js/storage.js`
`person_drawer.gd`	Disegna l'operaio col giubbotto catarifrangente (palette di colori inclusa).	`js/draw.js`
`game.gd`	Il gioco: nastro, giocatore, casse, colleghi IA, punteggio, disegno, fine turno.	`js/game.js`

Come si collegano: `game.gd` chiede le parole a `WordGen`, registra i progressi in `SaveManager` e disegna gli operai tramite `PersonDrawer`. `SaveManager` a sua volta usa `WordGen` per validare i file importati e `PersonDrawer` per i limiti delle palette.

Parte 1 · word_gen.gd — il generatore dei rifiuti (le parole)

Questo è il cuore "procedurale" del gioco. Non disegna nulla e non ha scene: è solo logica pura. Dato un seme (una stringa come "K7M2QX") e un numero d'ordine (0, 1, 2…) restituisce SEMPRE la stessa parola. È l'equivalente del file js/core.js della versione web.

N°	Codice	Spiegazione
1	`class_name WordGen`	Dà a questo script un nome globale: da qualsiasi altro script puoi scrivere WordGen.make_word(...) senza caricare il file a mano.
2	`extends RefCounted`	Ogni script GDScript estende una classe di Godot. RefCounted è la più leggera: oggetti che non stanno nella scena (niente nodo, niente disegno) e che Godot elimina da solo quando nessuno li usa più.
3	`## Costanti`	Commento: tutto ciò che segue il # viene ignorato da Godot, è solo una nota per chi legge (## è usato per le note "di documentazione").
4	`const CATS := [`	const = valore che non cambia mai. ":=" significa "deduci tu il tipo dal valore": qui un Array (lista). Le 7 categorie di rifiuto partono qui.
5	`{"name": "Plastica", "color": Color("#f5c518")},`	Un Dictionary (dizionario): coppie chiave: valore tra graffe. "color" è un Color di Godot, creato da un codice esadecimale HTML. La virgola finale separa dall'elemento successivo.
6	`{"name": "Carta", "color": Color("#3b82f6")},`	Carta = blu.
7	`{"name": "Vetro", "color": Color("#22c55e")},`	Vetro = verde.
8	`{"name": "Metallo", "color": Color("#cbd5e1")},`	Metallo = argento.
9	`{"name": "Legno", "color": Color("#b5733a")},`	Legno = marrone.
10	`{"name": "Organico", "color": Color("#ef4444")},`	Organico = rosso.
11	`{"name": "Indifferenziato", "color": Color("#c084fc")},`	Indifferenziato = viola.
12	`]`	Chiude la lista delle categorie. L'indice (0..6) di ogni elemento è il "numero di categoria" che useremo ovunque.
14	`const LENGTHS := [`	Le tre lunghezze possibili di una parola, con le loro regole.
15	`{"len": 4, "weight": 0.6, "base": 10, "carry": 1.0},`	4 lettere: 60% di probabilità (weight), vale 10 punti (base), non ti rallenta (carry = moltiplicatore di velocità 1.0).
16	`{"len": 6, "weight": 0.3, "base": 25, "carry": 0.75},`	6 lettere: 30% di probabilità, 25 punti, ti rallenta al 75%.
17	`{"len": 8, "weight": 0.1, "base": 60, "carry": 0.5},`	8 lettere: solo 10% (rare), 60 punti, ti rallenta al 50%.
18	`]`	Fine della lista.

N°	Codice	Spiegazione
20	`const CONS := "bbcccddfgglllmmmnnnpppprrrsssttttvvz"`	Le consonanti che il generatore può pescare. Le lettere comuni sono ripetute di proposito: pescando a caso una posizione della stringa, "r" o "t" escono più spesso di "z" (è un trucco per "pesare" le probabilità).
21	`const VOWS := "aaeeiiooou"`	Idem per le vocali: "a", "e", "i", "o" sono ripetute e la "u" è rara.
23	`const PATTERNS := {`	Per ogni lunghezza della METÀ della parola, i "modelli" possibili. Una parola da 4 lettere ha metà = 2; da 6, metà = 3; da 8, metà = 4.
24	`    2: ["CV", "CV", "CV", "VC", "VV"],`	Chiave 2 (le chiavi di un Dictionary possono essere numeri). C = consonante, V = vocale. "CV" è ripetuto per renderlo più frequente (suona più naturale: "Lu", "Ba"...).
25	`    3: ["CVC", "CVV", "VCV", "CVC", "CCV"],`	Modelli per metà di 3 lettere.
26	`    4: ["CVCV", "CVCC", "CVVC", "VCVC", "CCVC", "CVCV"],`	Modelli per metà di 4 lettere.
27	`}`	Chiude il dizionario.
29	`## Funzioni di utilità`	Altro commento di sezione.
30	`static func reversed_text(s: String) -> String:`	"static" = funzione che si chiama sulla classe (WordGen.reversed_text("ab")) senza creare un oggetto. Prende una String e ritorna ("->") una String. GDScript non ha un comando "reverse" per le stringhe, quindi lo scriviamo noi.
31	`    var out := ""`	var = variabile (può cambiare). Parte come stringa vuota; ":=" deduce il tipo String.
32	`    for i in range(s.length() - 1, -1, -1):`	range(inizio, fine, passo): parte dall'ultimo indice, scende di 1 alla volta e si ferma PRIMA di -1, cioè arriva a 0. Quindi legge la stringa al contrario.
33	`        out += s[i]`	s[i] è la lettera in posizione i (come stringa di un carattere). "+=" la attacca in fondo a out.
34	`    return out`	Restituisce la stringa rovesciata. ("abc" diventa "cba").
36	`static func is_palindrome(word: String) -> bool:`	Ritorna true/false: la parola si legge uguale da entrambi i lati? Serve anche per controllare i file importati.
37	`    var s := word.to_lower()`	Passa tutto in minuscolo, così "LuuL" e "luul" contano uguale.
38	`    return s == reversed_text(s)`	"==" confronta: vero se la parola è uguale alla sua versione rovesciata.
40	`static func category_of(text: String) -> int:`	Decide il tipo di rifiuto (0..6) di una parola. Dipende SOLO dalla parola, non dal seme: per questo il Pokédex è unico.
41	`    return text.to_lower().hash() % CATS.size()`	hash() trasforma la stringa in un numero grande ma sempre uguale per lo stesso testo. "%" è il resto della divisione: dà un numero da 0 a 6 (CATS.size() è 7).

N°	Codice	Spiegazione
43	`static func length_info(length: int) -> Dictionary:`	Dato 4, 6 o 8 ritorna il dizionario con peso, punti e velocità di trasporto.
44	`    for l in LENGTHS:`	Scorre le tre voci della lista.
45	`        if l["len"] == length:`	l["len"] legge la chiave "len" del dizionario.
46	`            return l`	Trovata: la restituisce e la funzione finisce subito.
47	`    return LENGTHS[0]`	Se per errore non trova nulla, ripiega sul 4 lettere (così non crasha).
49	`static func rng_for(seed_text: String, key: String) -> RandomNumberGenerator:`	Crea un generatore di numeri casuali "pilotato" dal seme + una chiave. Stesso seme e stessa chiave = stessa sequenza di numeri, sempre.
50	`    var rng := RandomNumberGenerator.new()`	Crea l'oggetto generatore. ".new()" costruisce una nuova istanza.
51	`    rng.seed = (seed_text + "|" + key).hash()`	Imposta il "seme interno" del generatore. Unisco seme e chiave con "|" e ne faccio l'hash: ogni coppia ha il suo flusso di numeri indipendente.
52	`    return rng`	Ritorna il generatore pronto all'uso.

La funzione principale: make_word

N°	Codice	Spiegazione
54	`static func make_word(seed_text: String, index: int) -> Dictionary:`	Crea la parola numero "index" del seme. Chiamandola due volte con gli stessi argomenti ottieni la stessa identica parola.
55	`    var rng := rng_for(seed_text, "w%d" % index)`	"w%d" % index è la formattazione stile printf: %d viene sostituito dal numero (es. "w7"). Ogni parola ha il suo generatore.
56	`    var r := rng.randf()`	randf() = numero decimale casuale tra 0 e 1.
57	`    var acc := 0.0`	Accumulatore delle probabilità: ci sommo i pesi uno alla volta.
58	`    var length := 4`	Lunghezza di partenza (se qualcosa va storto resta 4).
59	`    for l in LENGTHS:`	Scorre 4, 6, 8.
60	`        acc += l["weight"]`	Somma il peso: dopo il primo giro 0.6, dopo il secondo 0.9, dopo il terzo 1.0.
61	`        if r < acc:`	Se il numero estratto cade in questa "fetta" di probabilità…
62	`            length = l["len"]`	…sceglie questa lunghezza. Esempio: r = 0.75 è minore di 0.9 → 6 lettere.
63	`            break`	Esce dal ciclo: non serve controllare le altre.
64	`    var half := floori(length / 2.0)`	Metà della lunghezza come intero (4→2, 6→3, 8→4). Divido per 2.0 (decimale) e arrotondo per difetto con floori: evita l'avviso di "divisione tra interi".

N°	Codice	Spiegazione
65	`    var options: Array = PATTERNS[half]`	Prende la lista dei modelli per quella metà. ": Array" dichiara il tipo a mano perché il dizionario non lo conosce.
66	`    var pattern: String = options[rng.randi() % options.size()]`	randi() = intero casuale enorme; "%" con la quantità di modelli lo riduce a un indice valido. Pesca un modello, per esempio "CVC".
67	`    var half_text := ""`	Qui costruiamo la prima metà della parola.
68	`    for c in pattern:`	Scorre il modello lettera per lettera (le stringhe si possono iterare).
69	`        var source := CONS if c == "C" else VOWS`	Operatore ternario di GDScript: "valore se vero if condizione else valore se falso". Se il modello dice C usa le consonanti, altrimenti le vocali.
70	`        half_text += source[rng.randi() % source.length()]`	Pesca una lettera a caso da quel gruppo e la aggiunge alla metà.
71	`    var low := half_text + reversed_text(half_text)`	La magia del palindromo: metà + metà rovesciata. "lu" + "ul" = "luul".
72	`    var last := low.length() - 1`	Indice dell'ultima lettera (le posizioni partono da 0).
73	`    var text := low[0].to_upper() + low.substr(1, last - 1) + low[last].to_upper()`	Prima e ultima lettera maiuscole, quelle in mezzo restano com'erano. substr(da, quante) estrae la parte centrale: da posizione 1, per (last - 1) lettere. Risultato: "LuuL".
74	`    return {"text": text, "len": length, "cat": category_of(text)}`	Ritorna un dizionario con il testo, la lunghezza e il tipo di rifiuto (calcolato dalla parola).
76	`static func random_seed() -> String:`	Crea un seme casuale leggibile, da 6 caratteri (per il pulsante dadi).
77	`    var alphabet := "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"`	Lettere e cifre senza i caratteri ambigui (niente I, O, 0, 1) per non confonderli.
78	`    var out := ""`	Il seme che costruiamo.
79	`    for _i in 6:`	Ripete 6 volte. "for _i in 6" equivale a range(6); il trattino basso davanti dice a Godot "so che non uso questa variabile".
80	`        out += alphabet[randi() % alphabet.length()]`	randi() (senza rng) usa il generatore globale di Godot: qui il caso vero ci va bene, perché il seme deve essere imprevedibile.
81	`    return out`	Ritorna il seme.

Parte 2 · save_manager.gd — salvataggi, Pokédex, esporta/importa

Qui gestiamo i dati che devono sopravvivere alla chiusura del gioco: i salvataggi multipli (personaggio, seme, record) e il Pokédex generale. È un "Autoload": uno script che Godot carica da solo all'avvio e rende raggiungibile da ovunque col nome SaveManager. Si attiva da Progetto > Impostazioni progetto > Globali (Autoload), aggiungendo scripts/save_manager.gd con nome "SaveManager". È l'equivalente di js/storage.js, ma invece del localStorage del browser usa un file JSON nella cartella "user://" (una cartella privata del gioco, diversa per ogni sistema operativo).

N°	Codice	Spiegazione
1	`extends Node`	Un Autoload deve essere un Node, perché Godot lo aggiunge all'albero della scena all'avvio.
3	`const SAVE_PATH := "user://smista_rifiuti.json"`	Dove viene scritto il file di salvataggio. "user://" è la cartella dati dell'utente, sempre scrivibile.
4	`const FORMAT := "smista-rifiuti"`	Etichetta che mettiamo nei file esportati per riconoscere che sono davvero nostri.
6	`var slots: Array = []`	Lista dei salvataggi. Ogni salvataggio è un Dictionary: id, seed, character, stats, created, updated.
7	`var dex: Dictionary = {}`	Il Pokédex generale: chiave = parola, valore = {cat, len, n}. Unico per tutti i salvataggi e i semi.
8	`var last_id := ""`	Id dell'ultimo salvataggio giocato (per il pulsante "Continua").
9	`var mute := false`	Impostazione audio, salvata insieme al resto.
11	`func _ready() -> void:`	_ready() è chiamata da Godot una volta, quando il nodo è pronto. "-> void" = non restituisce nulla.
12	`    load_all()`	All'avvio carichiamo subito i dati dal disco.

Lettura e scrittura del file

N°	Codice	Spiegazione
14	`func load_all() -> void:`	Legge il file di salvataggio (se esiste) e riempie le variabili.
15	`    if not FileAccess.file_exists(SAVE_PATH):`	Se è la prima volta che si gioca il file non c'è ancora…
16	`        return`	…e usciamo: slots e dex restano vuoti.
17	`    var file := FileAccess.open(SAVE_PATH, FileAccess.READ)`	Apre il file in lettura. FileAccess è la classe di Godot per leggere e scrivere file.
18	`    var data = JSON.parse_string(file.get_as_text())`	Legge tutto il testo e lo trasforma in dati (Dictionary/Array). Non uso ":=" perché il tipo cambia a seconda del contenuto (se il file è rotto torna null).
19	`    file.close()`	Chiude il file: buona abitudine, libera il file subito.

N°	Codice	Spiegazione
20	`    if typeof(data) != TYPE_DICTIONARY:`	Se il risultato non è un dizionario il file è danneggiato…
21	`        return`	…e lo ignoriamo invece di far crashare il gioco.
22	`    slots = data.get("slots", [])`	get(chiave, valore_di_riserva): se la chiave manca usa la lista vuota.
23	`    dex = data.get("dex", {})`	Pokédex salvato, oppure vuoto.
24	`    last_id = data.get("last", "")`	Ultimo salvataggio usato.
25	`    mute = data.get("mute", false)`	Impostazione audio.
27	`func save_all() -> void:`	Scrive tutto su disco. La chiamiamo ogni volta che qualcosa di importante cambia.
28	`    var file := FileAccess.open(SAVE_PATH, FileAccess.WRITE)`	Apre (o crea) il file in scrittura, cancellando il contenuto precedente.
29	`    var data := {"slots": slots, "dex": dex, "last": last_id, "mute": mute}`	Impacchetta tutto in un unico dizionario.
30	`    file.store_string(JSON.stringify(data, "\t"))`	Converte il dizionario in testo JSON (con indentazione a tab, leggibile) e lo scrive.
31	`    file.close()`	Chiude e conferma la scrittura.

Gestione dei salvataggi (slot)

N°	Codice	Spiegazione
33	`func create_slot(character: Dictionary, seed_text: String) -> Dictionary:`	Crea un nuovo salvataggio con il personaggio e il seme scelti, e lo restituisce.
34	`    var clean_seed := seed_text.strip_edges().left(40)`	strip_edges() toglie gli spazi a inizio e fine; left(40) tronca a 40 caratteri.
35	`    if clean_seed == "":`	Se il giocatore ha lasciato il seme vuoto…
36	`        clean_seed = WordGen.random_seed()`	…ne generiamo uno casuale.
37	`    var now := Time.get_unix_time_from_system()`	Data/ora attuali come secondi dal 1970: comodo per ordinare.
38	`    var slot := {`	Costruisce il dizionario del nuovo salvataggio.
39	`        "id": "s%d%04d" % [int(now), randi() % 10000],`	Id unico: "s" + secondi + numero casuale a 4 cifre (%04d riempie di zeri). Con più valori, il formato vuole una lista [ ] dopo il %.
40	`        "seed": clean_seed,`	Il seme.
41	`        "character": character,`	Il personaggio (nome, sesso, capelli, colori).
42	`        "stats": {"best": 0, "saved": 0, "runs": 0},`	Statistiche: record, rifiuti smistati in totale, turni giocati.
43	`        "created": now,`	Quando è stato creato.

N°	Codice	Spiegazione
44	`"updated": now,`	Ultima modifica.
45	`}`	Fine del dizionario.
46	`slots.append(slot)`	Lo aggiunge alla lista.
47	`last_id = slot["id"]`	Diventa l'ultimo usato.
48	`save_all()`	Scrive su disco subito.
49	`return slot`	Restituisce il dizionario (è un riferimento: se lo modifichi, si modifica anche quello in lista).
51	`func get_slot(id: String) -> Dictionary:`	Cerca un salvataggio per id.
52	`for s in slots:`	Scorre tutti.
53	`if s["id"] == id:`	Trovato?
54	`return s`	Lo ritorna.
55	`return {}`	Altrimenti ritorna un dizionario vuoto: chi chiama controlla con .is_empty().
57	`func get_last_slot() -> Dictionary:`	Ritorna l'ultimo salvataggio giocato, o il più recente se non c'è.
58	`var s := get_slot(last_id)`	Prova con l'id memorizzato.
59	`if s.is_empty() and not slots.is_empty():`	Se non c'è ma esistono altri salvataggi...
60	`s = sorted_slots()[0]`	...prende il più recente.
61	`return s`	Ritorna il risultato (anche vuoto).
63	`func sorted_slots() -> Array:`	Copia della lista, dal più recente al più vecchio (per mostrarla nel menu).
64	`var copy := slots.duplicate()`	duplicate() copia la lista così non rimescoliamo quella vera.
65	`copy.sort_custom(func(a, b): return a["updated"] > b["updated"])`	sort_custom ordina con una regola scritta da noi. "func(a, b): ..." è una funzione anonima (lambda): ritorna true se a deve stare prima di b, cioè se è stato aggiornato più di recente.
66	`return copy`	Restituisce la lista ordinata.
68	`func save_slot(slot: Dictionary) -> void:`	Da chiamare quando finisci un turno o metti in pausa.
69	`slot["updated"] = Time.get_unix_time_from_system()`	Aggiorna la data di ultima modifica.
70	`last_id = slot["id"]`	Segna questo come ultimo giocato.
71	`save_all()`	Scrive su disco. (Non serve rimetterlo in lista: slot è già un riferimento all'elemento nella lista.)

N°	Codice	Spiegazione
73	`func remove_slot(id: String) -> void:`	Elimina un salvataggio.
74	`    slots = slots.filter(func(s): return s["id"] != id)`	filter tiene solo gli elementi per cui la lambda ritorna true: tutti tranne quello con quell'id.
75	`    if last_id == id:`	Se era l'ultimo usato…
76	`        last_id = ""`	…azzera il riferimento.
77	`    save_all()`	Salva la lista aggiornata.

Pokédex generale

N°	Codice	Spiegazione
79	`func add_to_dex(text: String) -> bool:`	Registra una parola smistata. Ritorna true se è NUOVA (mai trovata prima, in nessun seme).
80	`    if dex.has(text):`	has() controlla se la chiave esiste già.
81	`        dex[text]["n"] += 1`	Già presente: conta una volta in più.
82	`        return false`	Non è nuova.
83	`    dex[text] = {"cat": WordGen.category_of(text), "len": text.length(), "n": 1}`	Nuova voce: tipo, lunghezza e contatore a 1.
84	`    return true`	È nuova! Il gioco mostrerà il messaggio.

Esporta e importa

N°	Codice	Spiegazione
86	`func export_json(ids: Array = []) -> String:`	Produce il testo JSON da salvare in un file. Se "ids" è vuoto esporta tutti i salvataggi; "= []" è un valore predefinito.
87	`    var chosen: Array = []`	Lista dei salvataggi da esportare.
88	`    for s in slots:`	Passa tutti i salvataggi.
89	`        if ids.is_empty() or ids.has(s["id"]):`	Tutti se ids è vuoto, oppure solo quelli richiesti.
90	`            chosen.append(s)`	Aggiunge alla selezione.
91	`    var data := {"format": FORMAT, "version": 2, "dex": dex, "slots": chosen}`	Il file contiene l'etichetta, la versione, il Pokédex e i salvataggi scelti.
92	`    return JSON.stringify(data, "\t")`	Lo trasforma in testo.

N°	Codice	Spiegazione
94	`func export_to_file(path: String, ids: Array = []) -> bool:`	Scrive l'export in un file scelto dall'utente (il percorso lo dà un nodo FileDialog).
95	`    var file := FileAccess.open(path, FileAccess.WRITE)`	Apre il file di destinazione.
96	`    if file == null:`	Se non si può scrivere (percorso sbagliato, permessi)...
97	`        return false`	...diciamo che è fallita.
98	`    file.store_string(export_json(ids))`	Scrive il JSON.
99	`    file.close()`	Chiude.
100	`    return true`	Tutto ok.
102	`func import_from_file(path: String) -> int:`	Legge un file esportato. Ritorna quanti salvataggi ha importato (0 se il file non è valido).
103	`    var file := FileAccess.open(path, FileAccess.READ)`	Apre il file scelto.
104	`    if file == null:`	Impossibile aprirlo?
105	`        return 0`	Niente importato.
106	`    var text := file.get_as_text()`	Legge tutto.
107	`    file.close()`	Chiude.
108	`    return import_json(text)`	Passa il testo alla funzione che lo interpreta.
110	`func import_json(text: String) -> int:`	Interpreta il testo importato. Non ci fidiamo del file: controlliamo tutto prima di usarlo.
111	`    var data = JSON.parse_string(text)`	Prova a leggere il JSON (null se è invalido).
112	`    if typeof(data) != TYPE_DICTIONARY or data.get("format", "") != FORMAT:`	Deve essere un dizionario con la nostra etichetta.
113	`        return 0`	Altrimenti rifiutiamo il file.
114	`    merge_dex(data.get("dex", {}))`	Unisce il Pokédex del file con quello che hai già.
115	`    var count := 0`	Contatore dei salvataggi importati.
116	`    for raw in data.get("slots", []):`	Scorre i salvataggi presenti nel file.
117	`        var slot := sanitize_slot(raw)`	Li "ripulisce" e valida.
118	`        if not slot.is_empty():`	Se è utilizzabile...
119	`            slots.append(slot)`	...lo aggiunge come NUOVO salvataggio (mai sovrascrive quelli esistenti).
120	`            count += 1`	Conta.

N°	Codice	Spiegazione
121	`    save_all()`	Salva su disco.
122	`    return count`	Ritorna il numero importato.
124	`func sanitize_slot(raw) -> Dictionary:`	Ricostruisce un salvataggio partendo da dati non fidati. Nessun tipo dichiarato per "raw" perché potrebbe essere qualsiasi cosa.
125	`    if typeof(raw) != TYPE_DICTIONARY or typeof(raw.get("seed")) != TYPE_STRING:`	Serve un dizionario con un seme di tipo testo.
126	`        return {}`	Altrimenti scartato.
127	`    var ch: Dictionary = raw.get("character", {})`	Il personaggio scritto nel file, o vuoto.
128	`    var stats: Dictionary = raw.get("stats", {})`	Le statistiche scritte nel file.
129	`    var now := Time.get_unix_time_from_system()`	Ora attuale.
130	`    return {`	Costruiamo un salvataggio nuovo e pulito.
131	`        "id": "s%d%04d" % [int(now), randi() % 10000],`	Id nuovo: così un import non può mai sovrascrivere uno slot esistente.
132	`        "seed": str(raw["seed"]).strip_edges().left(40),`	Il seme, ripulito.
133	`        "character": sanitize_character(ch),`	Il personaggio, validato dalla funzione qui sotto.
134	`        "stats": {`	Statistiche con valori limitati tra 0 e un miliardo.
135	`            "best": clampi(int(stats.get("best", 0)), 0, 1000000000),`	int() perché JSON legge tutti i numeri come decimali (float); clampi li tiene entro i limiti.
136	`            "saved": clampi(int(stats.get("saved", 0)), 0, 1000000000),`	Come sopra, rifiuti smistati.
137	`            "runs": clampi(int(stats.get("runs", 0)), 0, 1000000000),`	Come sopra, turni giocati.
138	`        },`	Fine statistiche.
139	`        "created": now,`	Data di creazione = adesso.
140	`        "updated": now,`	Ultima modifica = adesso.
141	`    }`	Fine del dizionario ritornato.
143	`func sanitize_character(ch: Dictionary) -> Dictionary:`	Mette il personaggio in forma valida, qualunque cosa ci fosse nel file.
144	`    var hair := str(ch.get("hair", "short"))`	Il taglio di capelli come testo (str() converte qualsiasi cosa in String).
145	`    if not ["bald", "short", "long"].has(hair):`	Deve essere uno dei tre valori ammessi…
146	`        hair = "short"`	…altrimenti ripiega sul taglio corto.

N°	Codice	Spiegazione
147	`    return {`	Ritorna il personaggio pulito.
148	`        "name": str(ch.get("name", "Operaio")).left(14),`	Nome, massimo 14 caratteri.
149	`        "gender": "f" if ch.get("gender", "m") == "f" else "m",`	Sesso: solo "f" o "m".
150	`        "hair": hair,`	Capelli già validati.
151	`        "hair_color": clampi(int(ch.get("hair_color", 0)), 0, PersonDrawer.HAIR_COLORS.size() - 1),`	Indice del colore capelli, tenuto nei limiti della palette.
152	`        "shirt": clampi(int(ch.get("shirt", 0)), 0, PersonDrawer.SHIRTS.size() - 1),`	Indice del colore del giubbotto (15 colori).
153	`        "skin": clampi(int(ch.get("skin", 1)), 0, PersonDrawer.SKINS.size() - 1),`	Indice del colore della pelle (6 colori).
154	`    }`	Fine.
156	`func is_valid_word(text: String) -> bool:`	Controlla che una parola importata sia davvero una parola del nostro gioco.
157	`    var re := RegEx.new()`	RegEx = espressioni regolari, per controllare la forma di un testo.
158	`    re.compile("^[A-Za-z]{4,8}$")`	Solo lettere, da 4 a 8 caratteri (^ = inizio, $ = fine).
159	`    if re.search(text) == null:`	Se non rispetta la forma…
160	`        return false`	…non è valida.
161	`    if text.length() % 2 != 0:`	Le nostre parole hanno lunghezza pari (4, 6, 8).
162	`        return false`	Dispari = non valida.
163	`    if text[0] != text[0].to_upper() or text[text.length() - 1] != text[text.length() - 1].to_upper():`	Prima e ultima devono essere maiuscole.
164	`        return false`	Altrimenti non valida.
165	`    return WordGen.is_palindrome(text)`	Infine deve essere un palindromo.
167	`func merge_dex(src) -> void:`	Unisce un Pokédex esterno (da un file importato) nel nostro.
168	`    if typeof(src) != TYPE_DICTIONARY:`	Se non è un dizionario…
169	`        return`	…lo ignora.
170	`    for text in src:`	Scorrere un dizionario dà le CHIAVI, cioè le parole.
171	`        if not is_valid_word(str(text)):`	Scarta parole non valide.
172	`            continue`	continue = salta al giro successivo del ciclo.
173	`        var entry = src[text]`	Il valore associato alla parola.

N°	Codice	Spiegazione
174	`var n := 1`	Quante volte è stata smistata (almeno 1).
175	`if typeof(entry) == TYPE_DICTIONARY:`	Se il valore ha la forma giusta…
176	`n = maxi(1, int(entry.get("n", 1)))`	…legge il contatore (minimo 1).
177	`if dex.has(text):`	La parola c'è già nel nostro Pokédex?
178	`dex[text]["n"] = maxi(int(dex[text]["n"]), n)`	Tiene il numero più alto, non la somma (così importare due volte lo stesso file non gonfia i numeri).
179	`else:`	Altrimenti…
180	`dex[text] = {"cat": WordGen.category_of(text), "len": str(text).length(), "n": n}`	…crea la voce. Il tipo (colore) lo ricalcoliamo noi: non ci fidiamo di quello scritto nel file.

Parte 3 · person_drawer.gd — disegnare l'operaio

Nel gioco web i personaggi sono disegnati "a mano" con il canvas. Qui facciamo lo stesso con i comandi di disegno di Godot (draw_rect, draw_circle, draw_line...), senza immagini esterne. Importante: in Godot i comandi draw_* funzionano SOLO mentre un nodo sta eseguendo la sua funzione _draw(). Per questo la nostra funzione riceve come parametro il nodo (ci) su cui disegnare, e viene chiamata dal _draw() del gioco. È l'equivalente di js/draw.js.

N°	Codice	Spiegazione
1	`class_name PersonDrawer`	Nome globale: la useremo come PersonDrawer.draw_person(...).
2	`extends RefCounted`	Nessun nodo: è solo una "cassetta degli attrezzi" di funzioni statiche e costanti.
4	`const HAIR_COLORS := [Color("#1b1b1b"), Color("#4a2c17"), Color("#8b5a2b"), Color("#d9a441"), Color("#c0392b"), Color("#9aa0a6")]`	6 colori di capelli: nero, castano scuro, castano, biondo, rosso, grigio. Una lista di Color su una sola riga.
5	`const SKINS := [Color("#fde0c8"), Color("#f5cba7"), Color("#e0ac81"), Color("#c68642"), Color("#8d5524"), Color("#5c3a21")]`	6 tonalità di pelle, dalla più chiara alla più scura.
6	`const SHIRTS := [`	I 15 colori del giubbotto catarifrangente (è l'unica cosa che cambia colore).
7	`    Color("#e53935"), Color("#fb8c00"), Color("#fdd835"), Color("#7cb342"), Color("#2e7d32"),`	Rosso, arancione, giallo, verde chiaro, verde scuro.
8	`    Color("#00acc1"), Color("#1e88e5"), Color("#3949ab"), Color("#8e24aa"), Color("#d81b60"),`	Ciano, azzurro, indaco, viola, fucsia.
9	`    Color("#6d4c41"), Color("#f5f5f5"), Color("#9e9e9e"), Color("#37474f"), Color("#ff8a80"),`	Marrone, bianco, grigio, antracite, salmone.
10	`]`	Fine lista.
11	`const WORK_SHIRT := Color("#4b5563")`	Maglia da lavoro grigia, sotto il giubbotto (e delle maniche).
12	`const PANTS := Color("#2d3748")`	Colore dei pantaloni da lavoro.
13	`const REFLECTIVE := Color("#eef2f5")`	Colore delle strisce riflettenti del giubbotto.

La funzione di disegno

N°	Codice	Spiegazione
15	`static func draw_person(ci: CanvasItem, pos: Vector2, ch: Dictionary, o: Dictionary = {}) -> void:`	ci = il nodo su cui disegnare; pos = dove stanno i PIEDI; ch = il personaggio; o = opzioni facoltative (scala, direzione, camminata...). "= {}" è il valore predefinito.
16	`    var s: float = o.get("scale", 1.0)`	Fattore di ingrandimento (1.3 per il giocatore). get() prende il valore o quello di riserva.
17	`    var facing: float = o.get("facing", 1.0)`	1 = guarda a destra, -1 = a sinistra.

N°	Codice	Spiegazione
18	`var moving: bool = o.get("moving", false)`	Sta camminando?
19	`var carry: bool = o.get("carry", false)`	Sta portando un rifiuto? (braccia in avanti)
20	`var swing := 0.0`	Oscillazione delle gambe/braccia: 0 se fermo.
21	`if moving:`	Solo se cammina…
22	`swing = sin(o.get("phase", 0.0))`	…usa il seno della "fase" (che cresce nel tempo): oscilla avanti e indietro tra -1 e 1.
23	`var female: bool = ch["gender"] == "f"`	Vero se il personaggio è femmina (cambia busto, ciglia, orecchini, codino).
24	`var skin: Color = SKINS[int(ch["skin"])]`	Colore pelle scelto. int() perché dopo aver letto un JSON i numeri sono decimali e le liste vogliono indici interi.
25	`var hair_c: Color = HAIR_COLORS[int(ch["hair_color"])]`	Colore capelli.
26	`var vest: Color = SHIRTS[int(ch["shirt"])]`	Colore del giubbotto.
27	`var hair: String = ch["hair"]`	Taglio: "bald", "short" o "long".
29	`ci.draw_set_transform(pos, 0.0, Vector2(s * facing, s))`	Sposta l'origine del disegno ai piedi del personaggio (pos), senza rotazione, e applica la scala. Con facing = -1 la X viene specchiata: così disegniamo sempre "verso destra" e Godot lo gira per noi. Da qui in poi (0,0) = i piedi, e Y negativa = verso l'alto.
31	`if hair == "long":`	Capelli lunghi: vanno disegnati DIETRO al corpo, quindi per primi.
32	`ci.draw_rect(Rect2(-15, -85, 30, 50 if female else 40), hair_c)`	draw_rect(rettangolo, colore). Rect2(x, y, larghezza, altezza). Le femmine hanno i capelli più lunghi (50 contro 40).
34	`ci.draw_rect(Rect2(-10 + swing * 6.0, -26, 9, 26), PANTS)`	Gamba sinistra, spostata in avanti/indietro in base a swing.
35	`ci.draw_rect(Rect2(1 - swing * 6.0, -26, 9, 26), PANTS)`	Gamba destra: si muove in modo opposto (il segno meno) per sembrare una camminata.
36	`ci.draw_rect(Rect2(-12 + swing * 6.0, -4, 13, 5), Color.BLACK)`	Scarpa sinistra (nera) che segue la gamba.
37	`ci.draw_rect(Rect2(0 - swing * 6.0, -4, 13, 5), Color.BLACK)`	Scarpa destra.
39	`var body_w := 11.0 if female else 14.0`	Mezza larghezza del busto: più stretta per le femmine.
40	`ci.draw_rect(Rect2(-body_w, -58, body_w * 2.0, 32 if female else 34), WORK_SHIRT)`	Maglia da lavoro grigia: rettangolo centrato sullo 0.
41	`var vw := 9.0 if female else 12.0`	Mezza larghezza del giubbotto (poco più stretto della maglia, così restano visibili le maniche).
42	`ci.draw_rect(Rect2(-vw, -57, vw * 2.0, 30 if female else 33), vest)`	Il giubbotto colorato.

N°	Codice	Spiegazione
43	`ci.draw_rect(Rect2(-vw, -44, vw * 2.0, 4), REFLECTIVE)`	Prima striscia riflettente orizzontale.
44	`ci.draw_rect(Rect2(-vw, -33, vw * 2.0, 4), REFLECTIVE)`	Seconda striscia orizzontale.
45	`ci.draw_rect(Rect2(-vw + 3.0, -57, 3, 13), REFLECTIVE)`	Bretella verticale sinistra.
46	`ci.draw_rect(Rect2(vw - 6.0, -57, 3, 13), REFLECTIVE)`	Bretella verticale destra.
48	`for side in [-1.0, 1.0]:`	Disegna le due braccia con lo stesso codice: side = -1 (sinistra) e 1 (destra).
49	`var hand := Vector2(side * (body_w + 3.0) + side * swing * 2.0, -32.0 + side * swing * 5.0)`	Posizione della mano mentre cammina: ai fianchi, oscillando.
50	`if carry:`	Se porta un rifiuto…
51	`hand = Vector2(side * 21.0, -46.0)`	…le mani vanno in avanti, all'altezza del petto, ai lati del cartello.
52	`ci.draw_line(Vector2(side * body_w, -54), hand, WORK_SHIRT, 8.0)`	Il braccio: una linea spessa 8 dalla spalla alla mano, col colore della maglia da lavoro.
53	`ci.draw_circle(hand, 4.2, skin)`	La mano: un cerchietto color pelle.
55	`ci.draw_rect(Rect2(-4, -63, 8, 7), skin)`	Il collo.
56	`ci.draw_circle(Vector2(0, -73), 13.0, skin)`	La testa: cerchio di raggio 13.
57	`if hair != "bald":`	Se non è calvo…
58	`_half_disc(ci, Vector2(0, -75), 14.0, hair_c)`	…mette il "cappello" di capelli: un semicerchio sopra la testa (funzione definita sotto).
59	`if female:`	Dettagli femminili.
60	`if hair == "short":`	Capelli corti → codino.
61	`ci.draw_circle(Vector2(-17, -66), 6.0, hair_c)`	Il codino, dietro la nuca (in web era un'ellisse; qui un cerchio per semplicità).
62	`ci.draw_rect(Rect2(-16, -78, 5, 3), Color("#333333"))`	L'elastico.
63	`ci.draw_circle(Vector2(-12, -67), 2.2, Color("#f2c94c"))`	Un orecchino dorato.
65	`ci.draw_circle(Vector2(-1, -72), 1.8, Color("#222222"))`	Occhio sinistro (guardano verso destra, quindi sono spostati).
66	`ci.draw_circle(Vector2(7, -72), 1.8, Color("#222222"))`	Occhio destro.
67	`if female:`	Ciglia solo per le femmine.
68	`ci.draw_line(Vector2(-3, -74), Vector2(-6, -77), Color("#222222"), 1.8)`	Ciglia esterna dell'occhio sinistro.
69	`ci.draw_line(Vector2(-1, -75), Vector2(-2, -79), Color("#222222"), 1.8)`	Ciglia interna.

N°	Codice	Spiegazione
70	`        ci.draw_line(Vector2(9, -74), Vector2(12, -77), Color("#222222"), 1.8)`	Ciglia esterna dell'occhio destro.
71	`        ci.draw_line(Vector2(7, -75), Vector2(8, -79), Color("#222222"), 1.8)`	Ciglia interna.
72	`    var mouth := Color("#e0245e") if female else Color("#7a3b2e")`	Le labbra sono rosa per le femmine, marroni per i maschi.
73	`    ci.draw_arc(Vector2(3.5, -67), 3.0, 0.15 * PI, 0.85 * PI, 12, mouth, 3.0 if female else 1.5)`	Il sorriso: un arco di cerchio. Parametri: centro, raggio, angolo iniziale, angolo finale (in radianti), quanti segmenti usare, colore, spessore.
75	`    ci.draw_set_transform(Vector2.ZERO, 0.0, Vector2.ONE)`	Rimette la trasformazione "normale" (niente spostamento, niente scala): il nome non deve essere né ingrandito né specchiato.
76	`    var name_text: String = o.get("name", "")`	Il nome da scrivere sopra la testa (può mancare).
77	`    if name_text != "":`	Se c'è un nome...
78	`        var font := ThemeDB.fallback_font`	Prende il font predefinito di Godot (ThemeDB è il "database dei temi").
79	`        var p := pos + Vector2(-60, -94.0 * s)`	Dove scriverlo: sopra la testa. Il testo è centrato in un riquadro largo 120, quindi parte 60 pixel a sinistra.
80	`        ci.draw_string(font, p + Vector2(1, 1), name_text, HORIZONTAL_ALIGNMENT_CENTER, 120.0, 12, Color.BLACK)`	Prima una copia nera spostata di 1 pixel: fa da ombra/contorno e rende il nome leggibile su qualsiasi sfondo.
81	`        ci.draw_string(font, p, name_text, HORIZONTAL_ALIGNMENT_CENTER, 120.0, 12, Color.WHITE)`	Poi il nome vero, bianco. Parametri: font, posizione, testo, allineamento, larghezza del riquadro, dimensione, colore.

Funzione di supporto: il semicerchio

N°	Codice	Spiegazione
83	`static func _half_disc(ci: CanvasItem, center: Vector2, radius: float, color: Color) -> void:`	Disegna un semicerchio pieno rivolto verso l'alto. Il trattino basso iniziale è una convenzione: "funzione interna, non usarla da fuori".
84	`    var points := PackedVector2Array()`	PackedVector2Array è una lista compatta di punti: è quello che vuole draw_colored_polygon.
85	`    for i in range(0, 13):`	13 punti lungo la curva (da 0 a 12).
86	`        var a := PI + PI * float(i) / 12.0`	Angolo da PI a 2*PI (cioè la metà superiore: in Godot l'asse Y va verso il BASSO, quindi gli angoli con seno negativo stanno in alto).
87	`        points.append(center + Vector2(cos(a), sin(a)) * radius)`	Punto sul cerchio: coseno e seno danno la direzione, il raggio la distanza dal centro.

N°	Codice	Spiegazione
88	`ci.draw_colored_polygon(points, color)`	Riempie il poligono formato dai punti (la curva si chiude con una linea retta: il semicerchio).

Parte 4 · game.gd — il gioco vero e proprio

Questo è lo script del nodo principale (un Node2D). Fa tre cose, in questo ordine, ad ogni fotogramma: 1) aggiorna la logica (_process), 2) reagisce ai tasti (_unhandled_input), 3) ridisegna tutto (_draw). È l'equivalente di js/game.js. Per usarlo: crea una scena con un solo nodo Node2D, attacca questo script, salvala come main.tscn e impostala come scena principale del progetto.

N°	Codice	Spiegazione
1	`extends Node2D`	Node2D = nodo 2D con posizione, ed è il tipo giusto per disegnare con _draw().

Costanti del mondo

N°	Codice	Spiegazione
3	`const W := 960.0`	Larghezza dell'area di gioco in pixel. Uso numeri con la virgola (float) perché li mescolo con posizioni decimali.
4	`const H := 540.0`	Altezza dell'area di gioco.
5	`const WALL_H := 320.0`	Altezza del muro di fondo (sopra) prima che cominci il pavimento.
6	`const BELT_TOP := 430.0`	Y del bordo superiore del nastro trasportatore.
7	`const WALK_Y := 432.0`	Y dei piedi del giocatore (leggermente dietro al nastro).
8	`const BIN_TOP := 148.0`	Y in cui iniziano le casse.
9	`const BIN_X0 := 90.0`	X della prima cassa.
10	`const BIN_DX := 130.0`	Distanza tra una cassa e la successiva.
11	`const BIN_W := 110.0`	Larghezza di una cassa.
12	`const MAX_ERRORS := 15`	Con 15 errori il turno finisce.
13	`const HELPER_COST := [300, 1000, 2500]`	Punti necessari per assumere il 1°, 2° e 3° collega.
14	`const HELPER_NAMES := ["Gino", "Marta", "Pino", "Lia", "Otto", "Anna", "Ada", "Bob", "Ele", "Ivo"]`	Nomi tra cui sceglie il generatore.
15	`const PLAYER_SPEED := 260.0`	Velocità del giocatore in pixel al secondo.
16	`const HELPER_SPEED := 165.0`	Velocità dei colleghi (più lenti di te).
17	`const PLATE_SIZE := 22`	Dimensione del testo sui cartelli dei rifiuti.

Variabili di stato

N°	Codice	Spiegazione
19	`var slot: Dictionary = {}`	Il salvataggio in uso (seme, personaggio, statistiche).
20	`var font: Font`	Il font per scrivere. Lo assegniamo in _ready().
21	`var t := 0.0`	Secondi trascorsi nel turno.
22	`var level := 0`	"Livello di velocità": sale ogni 20 secondi.
23	`var belt_speed := 60.0`	Velocità attuale del nastro (pixel/secondo).
24	`var belt_offset := 0.0`	Scorrimento delle tacche del nastro (solo estetica).
25	`var score := 0`	Punti che hai A DISPOSIZIONE (si spendono per i colleghi).
26	`var spent := 0`	Punti spesi in colleghi: contano nel punteggio finale.
27	`var streak := 0`	Serie di rifiuti giusti consecutivi.
28	`var errors := 0`	Errori commessi.
29	`var saved := 0`	Rifiuti smistati correttamente in questo turno.
30	`var new_words := 0`	Parole nuove nel Pokédex trovate in questo turno.
31	`var spawn_index := 0`	Quante parole sono già arrivate: serve per pescare la prossima dal seme.
32	`var spawn_in := 1.2`	Secondi mancanti alla prossima parola.
33	`var next_id := 1`	Contatore per dare un id unico a ogni parola.
34	`var words: Array = []`	Le parole che stanno sul nastro (ognuna è un Dictionary: id, text, len, cat, x, claim).
35	`var player_x := W / 2.0`	Posizione orizzontale del giocatore (parte al centro).
36	`var player_facing := 1.0`	Direzione in cui guarda.
37	`var player_phase := 0.0`	Fase dell'animazione di camminata.
38	`var player_moving := false`	Si sta muovendo?
39	`var player_carry: Dictionary = {}`	La parola che porta in mano ({} = niente in mano).
40	`var helpers: Array = []`	I colleghi assunti (Dictionary ciascuno).
41	`var pops: Array = []`	Scritte che salgono e svaniscono ("+25", "-5").
42	`var toasts: Array = []`	Messaggi in alto ("Nuovo nel Pokédex: LuuL").
43	`var game_over := false`	True quando il turno è finito.
44	`var summary: Dictionary = {}`	Riepilogo da mostrare a fine turno.

Avvio

N°	Codice	Spiegazione
46	`func _ready() -> void:`	Chiamata una volta, appena il nodo entra in scena.
47	`    font = ThemeDB.fallback_font`	Prende il font predefinito.
48	`    process_mode = Node.PROCESS_MODE_ALWAYS`	Il nodo continua a ricevere input anche con il gioco in pausa: serve per poterlo rimettere in marcia premendo P.
49	`    _setup_inputs()`	Registra i tasti (funzione qui sotto).
50	`    slot = SaveManager.get_last_slot()`	Chiede all'Autoload l'ultimo salvataggio.
51	`    if slot.is_empty():`	Se non ne esiste nessuno (prima volta)…
52	`        var hero := {"name": "Niky", "gender": "m", "hair": "short", "hair_color": 1, "shirt": 1, "skin": 1}`	…usa un personaggio di prova. Nel gioco finito qui ci sarebbe la schermata di creazione.
53	`        slot = SaveManager.create_slot(hero, WordGen.random_seed())`	Crea un nuovo salvataggio con un seme casuale.
55	`func _setup_inputs() -> void:`	Crea i comandi da tastiera via codice, così non devi configurarli a mano nel progetto.
56	`    _add_action("move_left", [KEY_A, KEY_LEFT])`	Muovi a sinistra: tasto A o freccia sinistra.
57	`    _add_action("move_right", [KEY_D, KEY_RIGHT])`	Muovi a destra: D o freccia destra.
58	`    _add_action("act", [KEY_SPACE, KEY_E, KEY_DOWN])`	Prendi/lascia: Spazio, E o freccia giù.
59	`    _add_action("hire", [KEY_H])`	Assumi un collega: H.
60	`    _add_action("pause", [KEY_P, KEY_ESCAPE])`	Pausa: P o Esc.
62	`func _add_action(action: String, keys: Array) -> void:`	Crea un'"azione" (un nome come "act") e le associa dei tasti.
63	`    if not InputMap.has_action(action):`	Solo se non esiste già…
64	`        InputMap.add_action(action)`	…la crea. InputMap è il registro di tutti i comandi di Godot.
65	`    for k in keys:`	Per ogni tasto della lista…
66	`        var ev := InputEventKey.new()`	…crea un evento "tasto della tastiera"…
67	`        ev.physical_keycode = k`	…gli dice quale tasto è (physical = posizione fisica, indipendente dal layout, comodo per i tasti WASD)…
68	`        InputMap.action_add_event(action, ev)`	…e lo collega all'azione.

Input a evento singolo

N°	Codice	Spiegazione
70	`func _unhandled_input(event: InputEvent) -> void:`	Chiamata da Godot per ogni evento di input non già gestito da altri (es. da un pulsante dell'interfaccia).
71	`    if game_over:`	Se il turno è finito...
72	`        if event.is_action_pressed("ui_accept"):`	..."ui_accept" (Invio/Spazio) è un'azione già inclusa in Godot.
73	`            get_tree().paused = false`	Toglie l'eventuale pausa.
74	`            get_tree().reload_current_scene()`	Ricarica la scena = nuovo turno.
75	`        return`	In ogni caso non si fa altro a turno finito.
76	`    if event.is_action_pressed("pause"):`	P o Esc premuto...
77	`        get_tree().paused = not get_tree().paused`	...inverte la pausa. get_tree() è l'albero dei nodi; "paused" ferma i nodi che non sono impostati su "sempre attivi".
78	`    elif get_tree().paused:`	Se siamo in pausa...
79	`        return`	...ignoriamo tutti gli altri comandi.
80	`    elif event.is_action_pressed("act"):`	Spazio/E/freccia giù premuto (una sola volta, non ripetuto)...
81	`        _action()`	...prendi o lascia.
82	`    elif event.is_action_pressed("hire"):`	H premuto...
83	`        _hire()`	...prova ad assumere un collega.
84	`    elif event is InputEventMouseButton and event.pressed:`	Click del mouse o tocco (Godot converte i tocchi in click di default)...
85	`        if _hire_pill_rect().has_point(event.position):`	...se cade dentro il riquadro "assumi collega" (has_point verifica se il punto è nel rettangolo)...
86	`            _hire()`	...assume.

Il ciclo principale

N°	Codice	Spiegazione
88	`func _process(delta: float) -> void:`	Chiamata ad OGNI fotogramma. "delta" = secondi passati dal fotogramma precedente: moltiplicando le velocità per delta il gioco va alla stessa velocità su qualsiasi computer.
89	`    if game_over or get_tree().paused:`	Se è finito o in pausa...
90	`        queue_redraw()`	...ridisegniamo comunque (per mostrare le scritte)...
91	`        return`	...ma non aggiorniamo la logica.

N°	Codice	Spiegazione
92	`    t += delta`	Aggiunge il tempo trascorso.
93	`    level = int(t / 20.0)`	Ogni 20 secondi sale di un livello (int() taglia i decimali).
94	`    belt_speed = minf(190.0, 60.0 + level * 9.0)`	Il nastro accelera di 9 pixel/s per livello, con un massimo di 190. minf() = il più piccolo dei due.
95	`    belt_offset = fposmod(belt_offset + belt_speed * delta, 40.0)`	Fa scorrere le tacche. fposmod è il "resto" che non diventa mai negativo: l'offset resta tra 0 e 40 e il disegno si ripete.
96	`    _update_spawn(delta)`	Fa arrivare nuove parole.
97	`    _update_words(delta)`	Muove le parole sul nastro.
98	`    _update_player(delta)`	Muove il giocatore.
99	`    for i in helpers.size():`	Per ogni collega (usiamo l'indice perché serve a "prenotare" le parole)…
100	`        _update_helper(i, delta)`	…aggiorna la sua intelligenza artificiale.
101	`    _update_effects(delta)`	Fa svanire scritte e messaggi.
102	`    if errors >= MAX_ERRORS:`	Troppi errori?
103	`        _end_game()`	Fine del turno.
104	`    queue_redraw()`	Chiede a Godot di richiamare _draw() in questo fotogramma. Senza questa riga lo schermo non si aggiornerebbe.
106	`func _update_spawn(delta: float) -> void:`	Decide quando e quale parola far arrivare.
107	`    spawn_in -= delta`	Scala il conto alla rovescia.
108	`    if spawn_in > 0.0:`	Non è ancora ora…
109	`        return`	…esce.
110	`    var word := WordGen.make_word(slot["seed"], spawn_index)`	Pesca la parola numero "spawn_index" dal seme: stesso seme, stessa sequenza.
111	`    var rng := WordGen.rng_for(slot["seed"], "t%d" % spawn_index)`	Un secondo generatore, indipendente, per i tempi di arrivo.
112	`    spawn_index += 1`	La prossima volta pescherà la parola successiva.
113	`    spawn_in = maxf(0.9, 3.2 - level * 0.2) * (0.75 + rng.randf() * 0.5)`	Tempo fino alla prossima: parte da 3.2 s e scende col livello fino a 0.9 s; poi lo variamo del ±25% per non essere troppo regolare.
114	`    var width := _plate_w(word["text"])`	Larghezza del cartello di questa parola.
115	`    var x := W + width / 2.0 + 10.0`	Parte appena fuori dallo schermo a destra.
116	`    for o in words:`	Controlla le parole già sul nastro…

N°	Codice	Spiegazione
117	`x = maxf(x, o["x"] + (_plate_w(o["text"]) + width) / 2.0 + 14.0)`	...e si piazza dietro la più a destra, con 14 pixel di spazio, per non sovrapporsi.
118	`word["x"] = x`	Aggiunge la posizione al dizionario della parola.
119	`word["id"] = next_id`	Le dà un id unico (serve ai colleghi per "puntarla").
120	`word["claim"] = -1`	Nessun collega l'ha ancora prenotata (-1).
121	`next_id += 1`	Prepara l'id seguente.
122	`words.append(word)`	La mette sul nastro.
124	`func _update_words(delta: float) -> void:`	Fa scorrere le parole e gestisce quelle che cadono.
125	`for w in words.duplicate():`	Scorriamo una COPIA della lista, perché dentro il ciclo potremmo toglierne qualcuna (modificare una lista mentre la scorri crea errori).
126	`w["x"] -= belt_speed * delta`	Si muove verso sinistra.
127	`if w["x"] < 24.0:`	Ha superato la fine del nastro?
128	`words.erase(w)`	La toglie dal nastro.
129	`_penalty(w, "Persa!", w["x"] + 20.0, BELT_TOP)`	Perdi metà dei suoi punti.
131	`func _update_player(delta: float) -> void:`	Muove il giocatore in base ai tasti.
132	`var dir := Input.get_axis("move_left", "move_right")`	get_axis ritorna -1 (sinistra), 0 (niente) o 1 (destra) in base ai tasti premuti. Con un joystick darebbe anche valori intermedi.
133	`player_moving = dir != 0.0`	Si muove se la direzione non è zero.
134	`if player_moving:`	Se si muove...
135	`player_facing = signf(dir)`	...guarda nella direzione del movimento (signf = segno: -1 o 1).
136	`var mult := _carry_mult(player_carry)`	Più la parola è pesante, più va piano.
137	`player_x = clampf(player_x + dir * PLAYER_SPEED * mult * delta, 30.0, W - 30.0)`	Nuova posizione = vecchia + direzione × velocità × tempo. clampf impedisce di uscire dallo schermo.
138	`player_phase += delta * 11.0 * mult`	Fa avanzare l'animazione (più lenta se sei appesantito).
140	`func _carry_mult(carry: Dictionary) -> float:`	Moltiplicatore di velocità per ciò che si porta in mano.
141	`if carry.is_empty():`	Mani libere...
142	`return 1.0`	...velocità normale.
143	`return WordGen.length_info(carry["len"])["carry"]`	Altrimenti 1.0, 0.75 o 0.5 a seconda della lunghezza.

Prendere e lasciare i rifiuti

N°	Codice	Spiegazione
145	`func _plate_w(text: String) -> float:`	Larghezza del cartello: quella del testo più un po' di margine.
146	`    return font.get_string_size(text, HORIZONTAL_ALIGNMENT_LEFT, -1, PLATE_SIZE).x + 18.0`	get_string_size misura il testo col font e la dimensione dati; ".x" prende la larghezza.
148	`func _reachable(w: Dictionary, px: float) -> bool:`	Una parola è "a portata di mano" di chi sta in posizione px?
149	`    return absf(w["x"] - px) < _plate_w(w["text"]) / 2.0 + 26.0`	Sì se la distanza è minore di metà cartello più 26 pixel di tolleranza.
151	`func _bin_x(i: int) -> float:`	X del centro della cassa numero i.
152	`    return BIN_X0 + i * BIN_DX`	Prima cassa a 90, poi ogni 130 pixel.
154	`func _bin_under(x: float) -> int:`	Quale cassa sta sotto la posizione x? (-1 = nessuna)
155	`    for i in WordGen.CATS.size():`	Prova tutte le 7.
156	`        if absf(x - _bin_x(i)) < BIN_W / 2.0 + 4.0:`	Se x è dentro la larghezza della cassa (più 4 pixel)…
157	`            return i`	…ritorna il suo numero.
158	`    return -1`	Nessuna.
160	`func _action() -> void:`	Tasto "prendi/lascia".
161	`    if not player_carry.is_empty():`	Hai qualcosa in mano? Allora lo lasci.
162	`        var bin := _bin_under(player_x)`	Sei sotto una cassa?
163	`        if bin < 0:`	No…
164	`            _toast("Portala sotto una cassa!")`	…messaggio di aiuto…
165	`            return`	…e non fa altro.
166	`        var w := player_carry`	Si tiene da parte la parola…
167	`        player_carry = {}`	…svuota le mani…
168	`        _drop(w, bin, true)`	…e la butta (true = l'ha fatto il giocatore, quindi conta per la serie).
169	`    else:`	Mani vuote: allora raccogli.
170	`        var best: Dictionary = {}`	La parola più vicina trovata finora.
171	`        for w in words:`	Guarda tutte le parole sul nastro…
172	`            if _reachable(w, player_x) and (best.is_empty() or absf(w["x"] - player_x) < absf(best["x"] - player_x)):`	…se è a portata e (non ne abbiamo ancora una oppure è più vicina)…

N°	Codice	Spiegazione
173	`                best = w`	...diventa la migliore.
174	`            if not best.is_empty():`	Se ne ha trovata una...
175	`                words.erase(best)`	...la toglie dal nastro...
176	`                player_carry = best`	...e te la mette in mano.
178	`func _drop(w: Dictionary, bin: int, by_player: bool) -> void:`	Una parola viene buttata nella cassa "bin". Serve sia al giocatore che ai colleghi.
179	`    if bin != w["cat"]:`	Cassa sbagliata?
180	`        _penalty(w, "Cassa sbagliata!", _bin_x(bin), BIN_TOP + 20.0)`	Perdi metà punti.
181	`        return`	Fine: niente punti.
182	`    var pts: int = WordGen.length_info(w["len"])["base"]`	Punti base della parola (10, 25 o 60).
183	`    if by_player:`	Solo il giocatore...
184	`        streak += 1`	...aumenta la serie...
185	`        pts = roundi(pts * _multiplier())`	...e applica il moltiplicatore (roundi arrotonda all'intero).
186	`    score += pts`	Aggiunge i punti.
187	`    saved += 1`	Un rifiuto in più smistato.
188	`    slot["stats"]["saved"] += 1`	Statistica totale del salvataggio.
189	`    if SaveManager.add_to_dex(w["text"]):`	Registra nel Pokédex: se è nuova...
190	`        new_words += 1`	...la conta...
191	`        _toast("Nuovo nel Pokédex: " + w["text"])`	...e mostra il messaggio.
192	`    pops.append({"x": _bin_x(bin), "y": BIN_TOP, "text": "+%d" % pts, "color": Color("#7cfc9a"), "t": 0.0})`	Crea la scritta verde "+25" sopra la cassa: posizione, testo, colore e "t" (quanto è vecchia).
194	`func _multiplier() -> float:`	Moltiplicatore in base alla serie.
195	`    return 1.0 + minf(4.0, floorf(streak / 5.0)) * 0.5`	Ogni 5 giusti di fila +0.5, fino a ×3 (massimo 4 scatti). floorf arrotonda per difetto.
197	`func _penalty(w: Dictionary, why: String, x: float, y: float) -> void:`	Penalità per una parola persa o buttata male.
198	`    var pts := roundi(WordGen.length_info(w["len"])["base"] / 2.0)`	Metà dei punti che avrebbe valso.
199	`    score = maxi(0, score - pts)`	Sottrae, ma mai sotto zero (maxi = il più grande tra due interi).
200	`    streak = 0`	La serie si azzera.
201	`    errors += 1`	Un errore in più.

N°	Codice	Spiegazione
202	`    pops.append({"x": clampf(x, 40.0, W - 40.0), "y": y, "text": "%s -%d" % [why, pts], "color": Color("#ff6b6b"), "t": 0.0})`	Scritta rossa col motivo e i punti persi.

Colleghi controllati dall'intelligenza artificiale

N°	Codice	Spiegazione
204	`func _hire() -> void:`	Prova ad assumere un collega spendendo punti.
205	`    if helpers.size() >= HELPER_COST.size():`	Già al completo (3)?
206	`        _toast("La squadra è al completo!")`	Messaggio.
207	`        return`	Esce.
208	`    var cost: int = HELPER_COST[helpers.size()]`	Costo del prossimo: 300, poi 1000, poi 2500.
209	`    if score < cost:`	Punti insufficienti?
210	`        _toast("Servono %d punti per assumere un collega" % cost)`	Dice quanti ne servono.
211	`        return`	Esce.
212	`    score -= cost`	Paga.
213	`    spent += cost`	Ricorda quanto hai speso (conta nel punteggio finale).
214	`    var rng := WordGen.rng_for(slot["seed"], "helper%d" % helpers.size())`	Generatore per l'aspetto del collega: dallo stesso seme esce sempre lo stesso collega.
215	`    var look := {`	L'aspetto del collega, scelto a caso.
216	`        "name": "",`	Il nome serve solo alla funzione di disegno (lo mettiamo sotto).
217	`        "gender": "f" if rng.randf() < 0.5 else "m",`	Metà e metà.
218	`        "hair": ["bald", "short", "long"][rng.randi() % 3],`	Un taglio a caso tra tre.
219	`        "hair_color": rng.randi() % PersonDrawer.HAIR_COLORS.size(),`	Un colore capelli a caso.
220	`        "shirt": rng.randi() % PersonDrawer.SHIRTS.size(),`	Un colore di giubbotto a caso.
221	`        "skin": rng.randi() % PersonDrawer.SKINS.size(),`	Una pelle a caso.
222	`    }`	Fine dell'aspetto.
223	`    helpers.append({`	Aggiunge il collega alla squadra con tutto il suo stato.
224	`        "name": HELPER_NAMES[rng.randi() % HELPER_NAMES.size()],`	Nome a caso.
225	`        "look": look,`	Aspetto.

N°	Codice	Spiegazione
226	`"x": W - 40.0,`	Entra da destra.
227	`"carry": {},`	Mani vuote.
228	`"target": -1,`	Nessuna parola puntata (id -1).
229	`"wrong": false,`	Se sbaglierà cassa con la parola in mano.
230	`"facing": -1.0,`	Guarda a sinistra.
231	`"phase": 0.0,`	Animazione.
232	`"moving": false,`	Fermo.
233	`"cd": 0.0,`	"Cooldown": breve pausa dopo aver preso o lasciato qualcosa.
234	`})`	Fine del dizionario e della chiamata append.
235	`_toast("Nuovo collega: %s ti dà una mano!" % helpers[-1]["name"])`	helpers[-1] è l'ultimo elemento (l'appena assunto). Messaggio di benvenuto.
237	`func _update_helper(idx: int, delta: float) -> void:`	L'"intelligenza artificiale" di un collega: sceglie una parola, la raggiunge, la porta alla cassa giusta e la lascia.
238	`var h: Dictionary = helpers[idx]`	Prende il dizionario del collega (è un riferimento: le modifiche restano).
239	`var goal := -1.0`	Dove vuole andare (-1 = da nessuna parte).
240	`h["cd"] = maxf(0.0, h["cd"] - delta)`	Fa scendere il cooldown, senza andare sotto zero.
241	`var mult := _carry_mult(h["carry"])`	Anche i colleghi vanno più piano con le parole pesanti.
242	`if not h["carry"].is_empty():`	FASE 2: ha qualcosa in mano → va verso la cassa.
243	`var bin: int = h["carry"]["cat"]`	La cassa giusta per la sua parola.
244	`if h["wrong"]:`	Ogni tanto (8%) sbaglia…
245	`bin = (bin + 1) % WordGen.CATS.size()`	…e sceglie la cassa successiva.
246	`goal = _bin_x(bin)`	Il suo obiettivo è la X di quella cassa.
247	`if absf(h["x"] - goal) < 6.0 and h["cd"] <= 0.0:`	Arrivato (a meno di 6 pixel) e non in pausa?
248	`var w: Dictionary = h["carry"]`	Tiene da parte la parola…
249	`h["carry"] = {}`	…svuota le mani…
250	`h["cd"] = 0.4`	…fa una breve pausa…
251	`_drop(w, bin, false)`	…e la butta (false = non è il giocatore: niente serie).
252	`goal = -1.0`	Per ora non si muove più.

N°	Codice	Spiegazione
253	`    elif h["cd"] <= 0.0:`	FASE 1: mani vuote e non in pausa → cerca una parola.
254	`        var target := _word_by_id(h["target"])`	Controlla se la parola che aveva puntato è ancora sul nastro.
255	`        if target.is_empty() or target["claim"] != idx:`	Se non c'è più (qualcuno l'ha presa) oppure non è più sua…
256	`            h["target"] = -1`	…dimentica l'obiettivo…
257	`            target = {}`	…e ne cerca uno nuovo.
258	`            var best_time := INF`	INF = infinito: qualunque tempo reale sarà più piccolo.
259	`            for w in words:`	Guarda ogni parola sul nastro…
260	`                if w["claim"] != -1 and w["claim"] != idx:`	…saltando quelle già prenotate da un altro collega.
261	`                    continue`	Passa alla prossima.
262	`                var tt := _intercept_time(h["x"], w["x"])`	Quanto tempo ci metterebbe a raggiungerla?
263	`                if tt < best_time:`	La più "raggiungibile" vince.
264	`                    best_time = tt`	Ricorda il suo tempo…
265	`                    target = w`	…e la parola.
266	`            if not target.is_empty():`	Se ne ha trovata una…
267	`                target["claim"] = idx`	…la prenota col proprio numero…
268	`                h["target"] = target["id"]`	…e ne ricorda l'id.
269	`        if not target.is_empty():`	Ha un obiettivo?
270	`            goal = target["x"]`	Va verso la sua posizione (che cambia, perché il nastro scorre).
271	`            if _reachable(target, h["x"]) and absf(target["x"] - h["x"]) < 16.0:`	Abbastanza vicino per prenderla?
272	`                words.erase(target)`	La toglie dal nastro…
273	`                h["carry"] = target`	…se la mette in mano…
274	`                h["target"] = -1`	…non ha più un obiettivo…
275	`                h["cd"] = 0.25`	…breve pausa…
276	`                h["wrong"] = randf() < 0.08`	…e tira un dado: 8% di probabilità di sbagliare cassa.
277	`    h["moving"] = false`	Di base è fermo.
278	`    if goal >= 0.0 and (not h["carry"].is_empty() or h["cd"] <= 0.0):`	Si muove se ha un obiettivo e (porta qualcosa oppure non è in pausa).

N°	Codice	Spiegazione
279	`        var d: float = goal - h["x"]`	Distanza (con segno) dall'obiettivo.
280	`        if absf(d) > 4.0:`	Se è ancora lontano...
281	`            h["facing"] = signf(d)`	...si gira verso l'obiettivo...
282	`            h["x"] = clampf(h["x"] + signf(d) * minf(absf(d), HELPER_SPEED * mult * delta), 30.0, W - 30.0)`	...e avanza, ma mai più della distanza che resta (minf): così non "oltrepassa" l'obiettivo e non trema avanti e indietro.
283	`            h["moving"] = true`	Sta camminando.
284	`            h["phase"] += delta * 10.0`	Fa avanzare l'animazione.
286	`func _word_by_id(id: int) -> Dictionary:`	Cerca una parola sul nastro tramite il suo id.
287	`    for w in words:`	Scorre la lista.
288	`        if w["id"] == id:`	Trovata?
289	`            return w`	La ritorna.
290	`    return {}`	Altrimenti dizionario vuoto.
292	`func _intercept_time(hx: float, wx: float) -> float:`	Stima dopo quanti secondi un collega in posizione hx raggiunge una parola in wx (che scorre verso sinistra).
293	`    var dx := wx - hx`	Distanza con segno.
294	`    if dx >= 0.0:`	La parola è a destra del collega: viene verso di lui mentre lui va verso di lei...
295	`        return dx / (HELPER_SPEED + belt_speed)`	...le velocità si SOMMANO.
296	`    if HELPER_SPEED > belt_speed:`	La parola è a sinistra e lui la insegue: ce la fa solo se è più veloce del nastro...
297	`        return -dx / (HELPER_SPEED - belt_speed)`	...le velocità si SOTTRAGGONO.
298	`    return INF`	Altrimenti non la raggiungerà mai.

Effetti, messaggi e fine turno

N°	Codice	Spiegazione
300	`func _update_effects(delta: float) -> void:`	Fa invecchiare e sparire scritte e messaggi.
301	`    for p in pops:`	Per ogni scritta...
302	`        p["t"] += delta`	...invecchia...
303	`        p["y"] -= 28.0 * delta`	...e sale di 28 pixel al secondo.
304	`    pops = pops.filter(func(p): return p["t"] < 1.1)`	Tiene solo quelle più giovani di 1,1 secondi.

N°	Codice	Spiegazione
305	`    for m in toasts:`	Per ogni messaggio…
306	`        m["t"] += delta`	…invecchia.
307	`    toasts = toasts.filter(func(m): return m["t"] < 3.0)`	Tiene solo quelli più giovani di 3 secondi.
309	`func _toast(text: String) -> void:`	Mostra un messaggio in alto.
310	`    toasts.append({"text": text, "t": 0.0})`	Lo aggiunge con età zero.
311	`    if toasts.size() > 3:`	Massimo 3 messaggi contemporanei…
312	`        toasts.pop_front()`	…se sono troppi toglie il più vecchio.
314	`func _end_game() -> void:`	Chiude il turno e salva le statistiche.
315	`    game_over = true`	Ferma il gioco.
316	`    var total := score + spent`	Punteggio finale: i punti spesi per i colleghi contano.
317	`    var record: bool = total > int(slot["stats"]["best"])`	Hai battuto il record? (int() perché dopo il JSON i numeri sono decimali).
318	`    if record:`	Se sì…
319	`        slot["stats"]["best"] = total`	…aggiorna il record.
320	`    slot["stats"]["runs"] += 1`	Un turno in più giocato.
321	`    SaveManager.save_slot(slot)`	Scrive su disco (anche il Pokédex).
322	`    summary = {"score": total, "saved": saved, "new_words": new_words, "record": record, "best": int(slot["stats"]["best"])}`	Prepara il riepilogo da mostrare a schermo.

Disegno — _draw()

N°	Codice	Spiegazione
324	`func _draw() -> void:`	Godot la chiama quando serve ridisegnare (grazie a queue_redraw()). Qui, e solo qui, funzionano i comandi draw_*.
325	`    _draw_background()`	Muro e pavimento.
326	`    var carrying := not player_carry.is_empty()`	Il giocatore ha qualcosa in mano?
327	`    var under := _bin_under(player_x) if carrying else -1`	Se sì, quale cassa ha sotto i piedi (verrà evidenziata).
328	`    for i in WordGen.CATS.size():`	Disegna le 7 casse…
329	`        _draw_bin(i, i == under)`	…evidenziando quella sotto di te.
330	`    for h in helpers:`	Disegna i colleghi…

N°	Codice	Spiegazione
331	`        var opts := {"scale": 1.2, "facing": h["facing"], "phase": h["phase"], "moving": h["moving"], "carry": not h["carry"].is_empty(), "name": h["name"]}`	...con le loro opzioni di disegno.
332	`        PersonDrawer.draw_person(self, Vector2(h["x"], WALK_Y - 2.0), h["look"], opts)`	"self" = questo nodo, quello che sta eseguendo _draw().
333	`    var me := {"scale": 1.3, "facing": player_facing, "phase": player_phase, "moving": player_moving, "carry": carrying, "name": slot["character"]["name"]}`	Opzioni di disegno del giocatore (leggermente più grande dei colleghi).
334	`    PersonDrawer.draw_person(self, Vector2(player_x, WALK_Y), slot["character"], me)`	Disegna il giocatore.
335	`    _draw_belt()`	Il nastro, DAVANTI ai piedi (si disegna dopo = sta sopra).
336	`    var near: Dictionary = {}`	La parola che puoi prendere adesso (verrà evidenziata).
337	`    if not carrying:`	Solo a mani vuote...
338	`        for w in words:`	...cerca la più vicina a portata.
339	`            if _reachable(w, player_x) and (near.is_empty() or absf(w["x"] - player_x) < absf(near["x"] - player_x)):`	Stessa regola di _action().
340	`                near = w`	Trovata.
341	`    for w in words:`	Disegna i cartelli sul nastro.
342	`        _draw_plate(w["text"], w["cat"], Vector2(w["x"], BELT_TOP + 8.0), w == near)`	L'ultimo parametro è true solo per quella evidenziata.
343	`    for h in helpers:`	Cartelli portati dai colleghi.
344	`        if not h["carry"].is_empty():`	Solo se hanno qualcosa in mano.
345	`            _draw_plate(h["carry"]["text"], h["carry"]["cat"], Vector2(h["x"], WALK_Y - 60.0), false)`	Sul petto.
346	`    if carrying:`	Cartello portato dal giocatore.
347	`        _draw_plate(player_carry["text"], player_carry["cat"], Vector2(player_x, WALK_Y - 60.0), false)`	Sul petto.
348	`    _draw_effects()`	Scritte che salgono.
349	`    _draw_hud()`	Punteggi in alto.
350	`    if game_over:`	A fine turno...
351	`        _draw_game_over()`	...il riquadro finale.

N°	Codice	Spiegazione
353	`func _text(txt: String, center_x: float, baseline_y: float, size: int, color: Color) -> void:`	Scorciatoia per scrivere testo CENTRATO su un punto.
354	`    draw_string(font, Vector2(center_x - 300.0, baseline_y), txt, HORIZONTAL_ALIGNMENT_CENTER, 600.0, size, color)`	draw_string centra il testo dentro un riquadro largo 600 che parte 300 pixel a sinistra del centro desiderato. baseline_y è la linea su cui "poggiano" le lettere.
356	`func _draw_background() -> void:`	Sfondo.
357	`    draw_rect(Rect2(0, 0, W, WALL_H), Color("#34404f"))`	Muro: un rettangolo blu-grigio dall'alto fino a WALL_H.
358	`    draw_rect(Rect2(0, WALL_H, W, H - WALL_H), Color("#3d414b"))`	Pavimento: dal muro fino in fondo.
359	`    for x in range(0, int(W), 60):`	range(0, 960, 60): una riga ogni 60 pixel.
360	`        draw_line(Vector2(x, 0), Vector2(x, WALL_H), Color(1, 1, 1, 0.05), 2.0)`	Righe verticali quasi trasparenti (alpha 0.05) che danno l'effetto "pannelli del muro".
361	`    draw_rect(Rect2(0, WALL_H - 8.0, W, 8.0), Color(0, 0, 0, 0.25))`	Una fascia scura (zoccolo) in fondo al muro.
363	`func _draw_bin(i: int, active: bool) -> void:`	Disegna la cassa numero i.
364	`    var c: Color = WordGen.CATS[i]["color"]`	Colore della categoria.
365	`    var x := _bin_x(i)`	Centro della cassa.
366	`    var bot := BIN_TOP + 122.0`	Y del fondo.
367	`    var body := PackedVector2Array([Vector2(x - 50.0, BIN_TOP + 16.0), Vector2(x + 50.0, BIN_TOP + 16.0), Vector2(x + 42.0, bot), Vector2(x - 42.0, bot)])`	Il corpo è un trapezio (più largo in alto): 4 punti.
368	`    draw_colored_polygon(body, c)`	Lo riempie col colore della categoria.
369	`    for k in [-1, 0, 1]:`	Tre scanalature verticali per sembrare un bidone.
370	`        draw_rect(Rect2(x + k * 24.0 - 3.0, BIN_TOP + 30.0, 6.0, bot - BIN_TOP - 42.0), Color(0, 0, 0, 0.18))`	Strisce scure semitrasparenti.
371	`    var lid_y := BIN_TOP - 4.0 if active else BIN_TOP + 2.0`	Il coperchio si alza un po' se la cassa è evidenziata (sembra "aprirsi").
372	`    draw_rect(Rect2(x - 56.0, lid_y, 112.0, 16.0), c.darkened(0.25))`	Il coperchio: un colore più scuro (darkened) della cassa.
373	`    draw_rect(Rect2(x - 36.0, lid_y + 12.0, 72.0, 4.0), Color("#10141a"))`	La fessura scura.
374	`    if active:`	Se evidenziata…
375	`        draw_rect(Rect2(x - 58.0, BIN_TOP - 6.0, 116.0, 134.0), Color.WHITE, false, 2.0)`	…un contorno bianco (il "false" dice: non riempire, solo bordo; 2.0 è lo spessore).
376	`    _text(WordGen.CATS[i]["name"], x, bot + 18.0, 13, Color.WHITE if active else c)`	Il nome sotto la cassa: bianco se evidenziata, altrimenti del suo colore.

N°	Codice	Spiegazione
378	`func _draw_belt() -> void:`	Il nastro trasportatore.
379	`    for x in range(60, int(W), 220):`	Ogni 220 pixel...
380	`        draw_rect(Rect2(x, BELT_TOP + 30.0, 14.0, H - BELT_TOP - 30.0), Color("#17191e"))`	...una gamba che lo sostiene.
381	`    draw_rect(Rect2(0, BELT_TOP, W, 36.0), Color("#2d3138"))`	Il piano del nastro.
382	`    draw_rect(Rect2(0, BELT_TOP, W, 5.0), Color("#3a3f48"))`	Il bordo luminoso in alto.
383	`    var x := -belt_offset`	Le tacche partono da -offset: spostandosi a sinistra danno l'effetto di nastro che scorre.
384	`    while x < W:`	Finché siamo dentro lo schermo...
385	`        draw_rect(Rect2(x, BELT_TOP + 8.0, 3.0, 24.0), Color(0, 0, 0, 0.35))`	...disegna una tacca...
386	`        x += 40.0`	...e passa alla successiva, 40 pixel più in là.
387	`    draw_rect(Rect2(0, BELT_TOP - 30.0, 60.0, 70.0), Color(0.86, 0.15, 0.15, 0.35))`	La zona rossa a sinistra: dove le parole cadono nello scarto.
388	`    draw_string(font, Vector2(6, BELT_TOP + 56.0), "< SCARTO", HORIZONTAL_ALIGNMENT_LEFT, -1, 11, Color("#9ca3af"))`	L'etichetta della zona di scarto.
390	`func _draw_plate(text: String, cat: int, center: Vector2, highlight: bool) -> void:`	Disegna il "cartello" di una parola, colorato come il suo rifiuto.
391	`    var w := _plate_w(text)`	Larghezza in base al testo.
392	`    var col: Color = WordGen.CATS[cat]["color"]`	Colore del rifiuto.
393	`    var r := Rect2(center.x - w / 2.0, center.y - 16.0, w, 32.0)`	Il rettangolo del cartello, centrato su "center" (altezza 32).
394	`    draw_rect(r, Color("#0f1318"))`	Sfondo scuro: le parole colorate si leggono bene.
395	`    draw_rect(r, Color.WHITE if highlight else col.darkened(0.25), false, 3.0 if highlight else 2.0)`	Bordo: bianco e spesso se evidenziato, altrimenti del colore del rifiuto un po' scurito.
396	`    draw_string(font, Vector2(r.position.x, center.y + 8.0), text, HORIZONTAL_ALIGNMENT_CENTER, w, PLATE_SIZE, col)`	La parola, centrata nel cartello e colorata. Il +8 sposta la linea di base in modo che il testo sia al centro verticale.
398	`func _draw_effects() -> void:`	Scritte che salgono e svaniscono.
399	`    for p in pops:`	Per ogni scritta...
400	`        var a := clampf((1.1 - p["t"]) * 2.0, 0.0, 1.0)`	...l'opacità: 1 all'inizio, scende a 0 verso la fine della vita.
401	`        var c: Color = p["color"]`	Il suo colore...

N°	Codice	Spiegazione
402	`        c.a = a`	...con quella trasparenza (".a" = canale alpha).
403	`        _text(p["text"], p["x"], p["y"], 18, c)`	La scrive.
405	`func _hire_pill_rect() -> Rect2:`	Il rettangolo del riquadro "assumi collega" (usato per disegnarlo e per il click).
406	`    return Rect2(10, 46, 290, 26)`	Posizione (10, 46), dimensione 290×26.
408	`func _draw_hud() -> void:`	Barra in alto con le informazioni.
409	`    draw_rect(Rect2(0, 0, W, 40), Color(0.03, 0.04, 0.05, 0.62))`	Fascia scura semitrasparente.
410	`    var mult := _multiplier()`	Moltiplicatore attuale.
411	`    var err_col := Color("#ff6b6b") if errors > MAX_ERRORS - 5 else Color("#e5e7eb")`	Gli errori diventano rossi quando mancano meno di 5 al limite.
412	`    draw_string(font, Vector2(14, 27), "Punti %d" % score, HORIZONTAL_ALIGNMENT_LEFT, -1, 18, Color.WHITE)`	Punti disponibili.
413	`    draw_string(font, Vector2(160, 27), "Serie %d ×%.1f" % [streak, mult], HORIZONTAL_ALIGNMENT_LEFT, -1, 18, Color("#ffd166") if mult > 1.0 else Color("#9ca3af"))`	Serie e moltiplicatore (%.1f = un decimale). Dorato quando il moltiplicatore è attivo.
414	`    draw_string(font, Vector2(320, 27), "Velocità %d" % (level + 1), HORIZONTAL_ALIGNMENT_LEFT, -1, 18, Color("#9ad1ff"))`	Livello di velocità.
415	`    draw_string(font, Vector2(460, 27), "Errori %d/%d" % [errors, MAX_ERRORS], HORIZONTAL_ALIGNMENT_LEFT, -1, 18, err_col)`	Errori.
416	`    draw_string(font, Vector2(610, 27), "Colleghi %d" % helpers.size(), HORIZONTAL_ALIGNMENT_LEFT, -1, 18, Color("#c4b5fd"))`	Numero di colleghi.
417	`    draw_string(font, Vector2(W - 10.0 - 300.0, 26), "Seme: " + str(slot["seed"]), HORIZONTAL_ALIGNMENT_RIGHT, 300.0, 14, Color("#9ca3af"))`	Il seme, allineato a destra in un riquadro largo 300 che finisce 10 pixel prima del bordo.
419	`    var r := _hire_pill_rect()`	Il riquadro per assumere un collega.
420	`    draw_rect(r, Color(0.03, 0.04, 0.05, 0.7))`	Sfondo scuro.
421	`    if helpers.size() < HELPER_COST.size():`	Se puoi ancora assumere...
422	`        var cost: int = HELPER_COST[helpers.size()]`	...il costo del prossimo.
423	`        var can := score >= cost`	Hai abbastanza punti?
424	`        var fill := clampf(float(score) / float(cost), 0.0, 1.0)`	Quanta parte del prezzo hai già: da 0 a 1 (divido float per float, altrimenti sarebbe una divisione tra interi e darebbe 0).
425	`        var green := Color("#22c55e") if int(t * 2.5) % 2 == 0 else Color("#16a34a")`	Verde che lampeggia: cambia tono ogni 0,4 secondi.

N°	Codice	Spiegazione
426	`draw_rect(Rect2(r.position, Vector2(r.size.x * fill, r.size.y)), green if can else Color("#3b4a63"))`	Barra di avanzamento: la parte riempita è larga quanto "fill" della larghezza. Verde se puoi comprare, azzurro-grigio altrimenti.
427	`var label := "ASSUMI collega %d (-%d)  [H]" % [helpers.size() + 1, cost] if can else "Collega %d: %d/%d punti" % [helpers.size() + 1, score, cost]`	Il testo cambia: se puoi, ti dice di premere H; altrimenti mostra a quanti punti sei.
428	`_text(label, r.position.x + r.size.x / 2.0, r.position.y + 18.0, 13, Color.WHITE)`	Scrive il testo centrato nel riquadro.
429	`else:`	Altrimenti sono già tutti assunti...
430	`_text("Squadra al completo", r.position.x + r.size.x / 2.0, r.position.y + 18.0, 13, Color.WHITE)`	...e lo dice.
432	`for i in toasts.size():`	I messaggi, uno sotto l'altro.
433	`var m: Dictionary = toasts[i]`	Il messaggio i-esimo.
434	`var tw := font.get_string_size(m["text"], HORIZONTAL_ALIGNMENT_LEFT, -1, 16).x + 24.0`	Larghezza del testo più il margine.
435	`var alpha := clampf(3.0 - m["t"], 0.0, 1.0)`	Svanisce nell'ultimo secondo.
436	`draw_rect(Rect2(W / 2.0 - tw / 2.0, 78.0 + i * 30.0, tw, 26.0), Color(0, 0, 0, 0.6 * alpha))`	Sfondo scuro del messaggio.
437	`_text(m["text"], W / 2.0, 96.0 + i * 30.0, 16, Color(1, 1, 1, alpha))`	Il testo bianco.
439	`func _draw_game_over() -> void:`	Riquadro di fine turno.
440	`draw_rect(Rect2(0, 0, W, H), Color(0, 0, 0, 0.6))`	Scurisce tutto lo schermo.
441	`var box := Rect2(W / 2.0 - 190.0, 120.0, 380.0, 300.0)`	Il riquadro centrale.
442	`draw_rect(box, Color("#1a212b"))`	Sfondo del riquadro.
443	`draw_rect(box, Color("#34404f"), false, 2.0)`	Bordo.
444	`_text("Turno finito!", W / 2.0, 165.0, 26, Color.WHITE)`	Titolo.
445	`if summary.get("record", false):`	Se hai fatto un nuovo record...
446	`_text("NUOVO RECORD!", W / 2.0, 200.0, 18, Color("#f5c518"))`	...lo scrive in oro.
447	`_text("Punti: %d  (record %d)" % [summary["score"], summary["best"]], W / 2.0, 245.0, 18, Color.WHITE)`	Punti e record.
448	`_text("Rifiuti smistati: %d" % summary["saved"], W / 2.0, 280.0, 18, Color.WHITE)`	Quanti ne hai smistati.

N°	Codice	Spiegazione
449	`    _text("Nuovi nel Pokédex: %d" % summary["new_words"], W / 2.0, 315.0, 18, Color.WHITE)`	Quanti nuovi hai scoperto.
450	`    _text("Premi Invio per ricominciare", W / 2.0, 385.0, 16, Color("#9ad1ff"))`	Istruzione finale (vedi _unhandled_input: ui_accept ricarica la scena).

8 · Cosa manca e come proseguire

Per tenere il codice leggibile, questa versione contiene il "motore" del gioco. Ecco cosa non è stato portato (sono ottimi esercizi, dal più facile al più difficile):

- **Suoni:** aggiungi un nodo `AudioStreamPlayer` e riproduci un suono in `_drop()` (corretto/sbagliato) e quando trovi una parola nuova. Rispetta `SaveManager.mute`.
- **Cambiare i numeri:** modifica `HELPER_COST`, `MAX_ERRORS`, i punti in `LENGTHS` e guarda come cambia il bilanciamento. Il modo più rapido per capire come tutto è collegato.
- **Animazione di lancio:** nella versione web la parola vola verso la cassa. Aggiungi una lista `fly` come `pops`, con posizione di partenza/arrivo e un tempo, e disegnala in `_draw()`.
- **Menu e creazione del personaggio:** si costruiscono con nodi `Control` (Button, LineEdit, OptionButton, ColorPickerButton, FileDialog) nell'editor; ogni pulsante invia un segnale (`pressed`) collegato a una funzione. Per l'anteprima si può riusare `PersonDrawer.draw_person` dentro un nodo Control.
- **Pokédex a schermo:** una scena con `GridContainer` o `ItemList` riempita leggendo `SaveManager.dex`, con filtri per tipo.
- **Salvare la partita a metà:** aggiungi una funzione che mette in un Dictionary tempo, punti, parole sul nastro e colleghi, e la salva nello slot (come faceva `snapshot()` nella versione web).
- **Touch:** usa `TouchScreenButton` collegati alle azioni "move_left", "move_right", "act", "hire".
- **Sostituire il disegno con sprite:** invece di `draw_rect` puoi usare `Sprite2D` e `AnimatedSprite2D`: è il passo naturale quando avrai grafica vera.

Parole chiave utili da cercare nella documentazione di Godot: Node2D, CanvasItem._draw, InputMap, FileAccess, JSON, RandomNumberGenerator, Autoload, signals.